Pug Coding example

1. **Pre-requisiti**: Node.js e npm: Assicurati di avere installato Node.js (che include npm). Puoi scaricarlo da nodejs.org.

Node.js: è un ambiente di esecuzione JavaScript lato server, che permette di eseguire codice JavaScript fuori dal browser per creare applicazioni scalabili e performanti.

npm: È il package manager per Node.js, che consente di installare, gestire e condividere librerie e pacchetti di codice in modo semplice ed efficiente.

2. **Creazione della cartella del progetto**: apri il terminale e crea una nuova cartella per il progetto, ad esempio:

```
mkdir progetto-pug
cd progetto-pug
```

3. **Inizializzazione del progetto con npm**: all'interno della cartella del progetto, esegui:

```
npm init -y
```

Questo comando crea un file package.json con le impostazioni di default.

4. **Installazione delle dipendenze**: possiamo utilizzare Pug come motore di template e Express per creare un semplice server web, installa i pacchetti necessari:

```
npm install express pug
```

5. **Creazione della struttura del progetto**: in questo modo.

```
progetto-pug/
├── views/
│   └── index.pug
├── index.js
└── package.json
```

6. Configurazione del server Node.js: crea un file chiamato index.js e inserisci il seguente codice:

```
const express = require('express');
const app = express();

// Imposta Pug come motore di template
app.set('view engine', 'pug');
app.set('views', './views');
// Definisci una route di esempio
app.get('/', (req, res) => {
  res.render('index', { title: 'Progetto Pug', message: 'Ciao, questo è un test con Pug!' });
});

// Avvia il server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server in esecuzione su http://localhost:${PORT}`);
});
```

7. **Creazione del template Pug**: nella cartella views, creiamo il file index.pug con il seguente contenuto.

```
doctype html
html
  head
    title=title
  body
    h1=message
```

8. **Avvio del progetto**. Nel terminale eseguiamo:

```
node index.js
```

Se tutto funziona correttamente, vedrai un messaggio nel terminale che indica che il server è in esecuzione su http://localhost:3000.

Approfondimento su variabili e funzioni nei CSS3.

Le variabili CSS, dette anche Custom Properties, permettono di definire valori riutilizzabili in tutto il foglio di stile.

Sintassi di definizione:

--nome-variabile: valore;

--custom-theme-brand-color: hsl(358, 100%, 48%);

Richiamo della variabile:
Per utilizzare una variabile, si impiega la funzione var():

color: var(--custom-theme-brand-color);

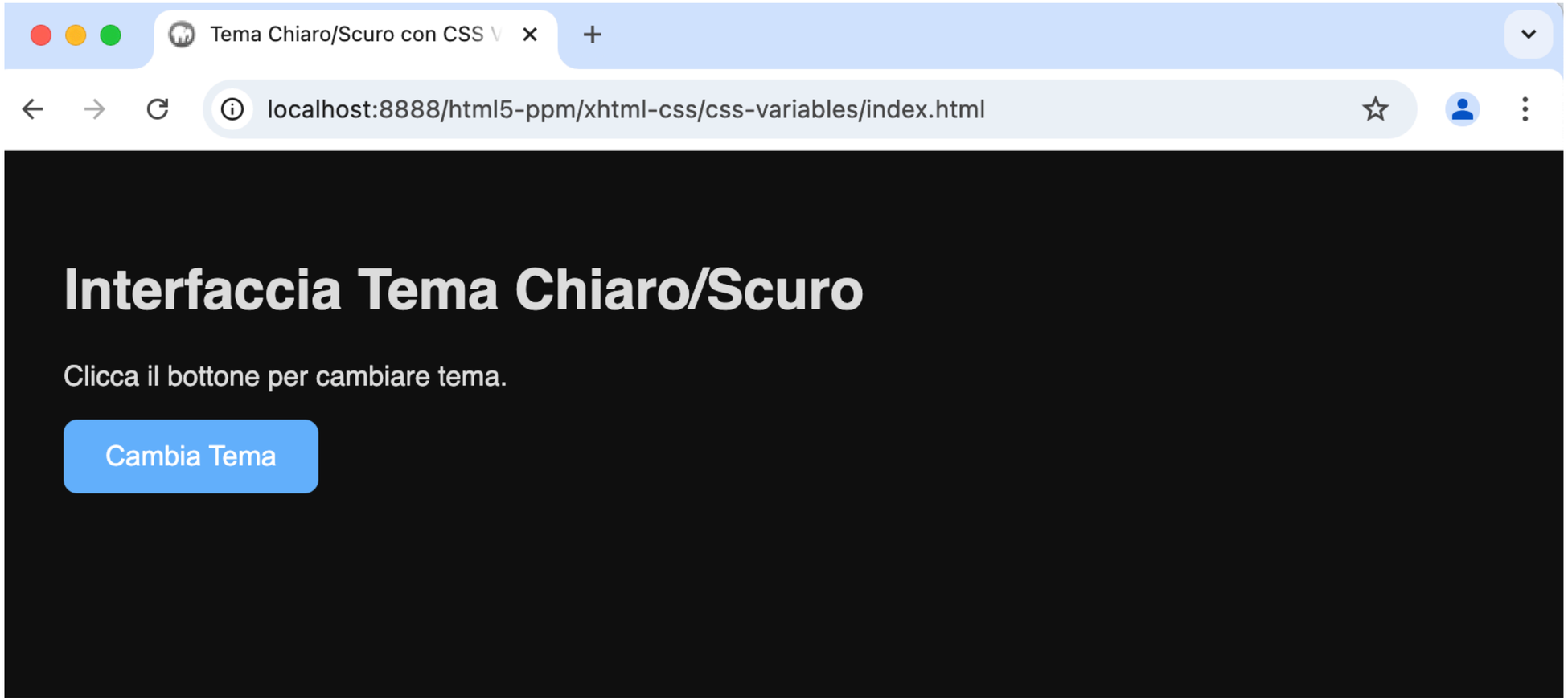
Vantaggi delle variabili CSS

- Centralizzazione dei valori: un solo punto di modifica per aggiornare colori, spaziature, font, ecc.
- Facilità di manutenzione: riduce la duplicazione del codice e rende più semplice il cambio del tema o degli stili in maniera dinamica.
- Ereditarietà: le variabili **definite a livello di root o di un selettore specifico si propagano ai figli**, consentendo una gestione gerarchica dei valori.

Esempio d'uso:

```
:root {
  --main-color: #0089c7;
}
```

```
div {
  color: var(--main-color)
  --div-color: #0089c7;
}
```



Esempio d'uso variabili css3: http://localhost:8888/html5-ppm/xhtml-css/css-variables/index.html

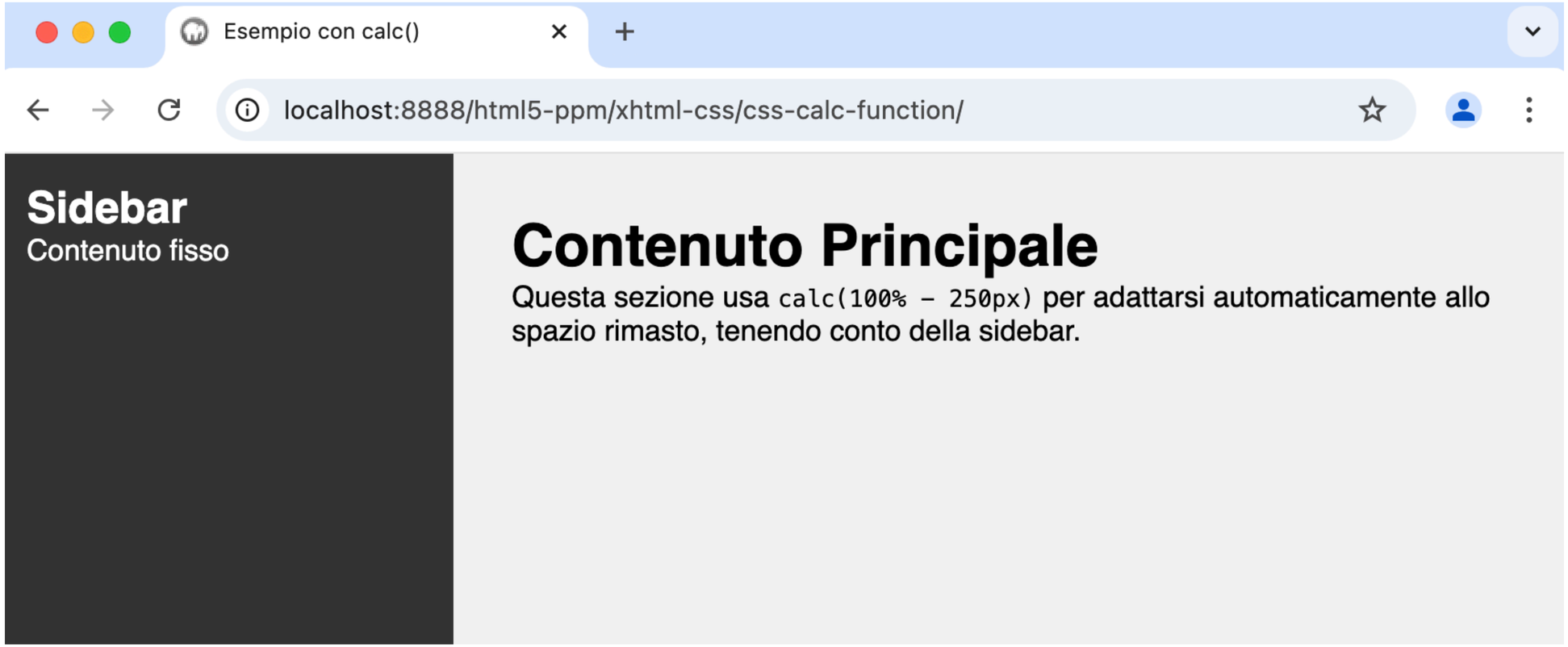
Funzioni CSS Utili

- La funzione var(nome var, fallback): è la funzione principale per utilizzare le variabili CSS, permettendo anche di definire un valore di fallback se la variabile non è stata definita.

- Funzione calc(): permette di eseguire operazioni matematiche direttamente in CSS per calcolare dimensioni, posizioni, ecc.

```
width: calc(100% - 2rem);
```

Utilizzo: fondamentale per creare layout fluidi e dinamici, specialmente quando combinati con altre unità di misura.

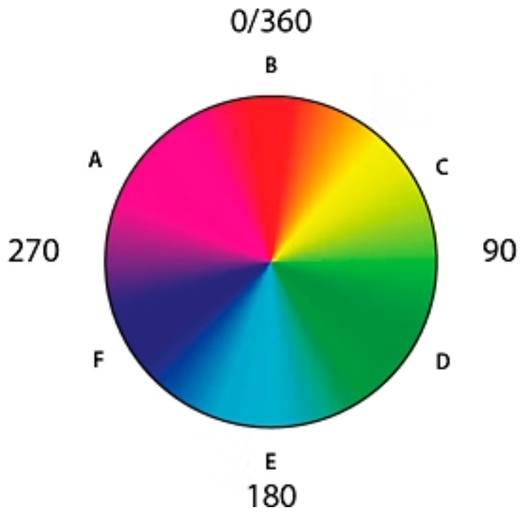


Esempio d'uso calc function: http://localhost:8888/html5-ppm/xhtml-css/css-calc-function/

Funzioni di colore:

- hsl() e hsla(): permettono di definire colori basati su tonalità, saturazione e luminosità. css

  color: hsl(358, 100%, 48%);



Funzioni di manipolazione del colore: alcune librerie o preprocessori (come Sass) offrono funzioni per schiarire, scurire o modificare il colore. In CSS puro, la **sintassi è più limitata**, ma l'uso di HSL semplifica la creazione di variazioni (es. il valore "shade").

Funzioni per i gradienti:

- linear-gradient() e radial-gradient(): consentono di creare sfumature lineari o radiali senza dover ricorrere a immagini esterne.

```
background-image: linear-gradient(0deg, rgba(0, 0, 0, 0.3), rgba(0, 0, 0, 0.3));
```

Utilizzo: perfette per effetti visivi moderni e per aggiungere profondità senza appesantire il caricamento della pagina.

Esempio d'uso linear-gradient: http://localhost:8888/html5-ppm/xhtml-css/css-linear-gradient/

Altre funzioni utili:

- rotate(), scale(), translate(): funzioni che fanno parte del modulo CSS Transforms per applicare trasformazioni agli elementi.

  transform: rotate(45deg);

  Esempi: http://localhost:8888/html5-ppm/css3/

Animazioni e Transizioni in CSS

Le transizioni permettono di aggiungere un effetto fluido quando una proprietà CSS cambia. Si usano con transition e funzionano su eventi come hover, focus o click.

```
button {
  transition: background-color 0.3s ease;
}
```

Le animazioni, invece, permettono di creare movimenti complessi e controllati nel tempo usando @keyframes.

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
```

Metalinguaggi: scss & sass (Pre-processors)

Sono metalinguaggi che aumentano le capacità del linguaggio css e che tuttavia hanno bisogno di una compilazione a css per essere utilizzabili nel browser

SCSS	Sass
Stands for Sassy CSS	Stands for Syntactically Awesome Style Sheets
Similar to CSS (uses curly braces and semicolons)	Uses strict indentation (like Python)
Any valid CSS is valid SCSS	CSS code cannot be used as SASS
`.scss` extension	`.sass` extension

Metalinguaggi: scss & sass

Sono metalinguaggi che aumentano le capacità del linguaggio css e che tuttavia hanno bisogno di una compilazione a css per essere utilizzabili nel browser

css

```
body {
  background-color:#000;
  color:#fff;
}
```

scss

```
body {
  background-color:#000;
  color:#fff;
}
```

sass

```
body
  background-color:#000;
  color:#fff;
```

Metalinguaggi: scss & sass

Nesting

css

```
nav {
  background-color:#333;
  padding:1em;
}
nav ul {
  margin:0;
  padding:0;
  list-style:none;
}
nav ul li {
  display:inline-block;
}
```

scss

```
nav {
  background-color:#333;
  padding:1em;
  ul {
    margin:0;
    padding:0;
    list-style:none;
    li {
      display:inline-block;
    }
  }
}
```

sass

```
nav
    background-color: #333
    padding: 1em

    ul
        margin: 0
        padding: 0
        list-style: none

    li
        display: inline-block
```

Metalinguaggi: scss & sass

& in Nesting

css

```
button {
  background-color: #535353;
  color: #000;
}
button:hover {
  background-color: #000;
  color: #fff;
}
```

scss

```
button {
  background-color: #535353;
  color: #000;
  &:hover {
    background-color: #000;
    color: #fff;
  }
}
```

sass

```
button
  background-color: #535353
  color: #000
  &:hover
      background-color: #000
      color: #fff
```

Metalinguaggi: scss & sass

Variables

css

```
body {
  font-family: Nunito,
  sans-serif;
  color: #ffd969;
}

body > div {
 width: 660px;

}
```

scss

```
$my-font: Nunito, sans-serif;
$my-font-color: #ffd969;
$content-width: 660px;

body {
  font-family: $my-font;
  color: $my-font-color;
  & > div {
    width: $content-width;
  }
}
```

sass

```
$my-font: Nunito, sans-serif
$my-font-color: #ffd969
$content-width: 660px

body
  font-family: $my-font
  color: $my-font-color

  & > div
    width: $content-width
```

Metalinguaggi: scss & sass

Variables: local vs global

css

```
body {
  font-family: Nunito,
  sans-serif;
  color: #ffd969;
}

body > div {
 width: 660px;
 color: #ff0000;
}
```

scss

```
$my-font: Nunito, sans-serif;
$my-font-color: #ffd969;
$content-width: 660px;

body {
  font-family: $my-font;
  color: $my-font-color;
  & > div {
    $my-font-color: #ff0000;
    width: $content-width;
    color: $my-font-color;
  }
}
```

sass

```
$my-font: Nunito, sans-serif
$my-font-color: #ffd969
$content-width: 660px

body
  font-family: $my-font
  color: $my-font-color

  & > div
    $my-font-color: #ff0000
    width: $content-width
    color: $my-font-color
```

Metalinguaggi: scss & sass

Functions: mixins

css

```
.element-1 {
  position: absolute;
  left: 50%;
  top: 50%;
 transform:
 translate(-50%, -50%);
}

.element-2 {
  position: absolute;
  left: 50%;
  top: 50%;
  transform:
   translate(-50%, -50%);
  color: blue;
}
```

scss

```
$position: 50%;

@mixin absolute-center($position) {
  position: absolute;
  left: $position;
  top: $position;
  transform: translate(-50%,-50%);
}

.element-1 {
  @include
   absolute-center($position);
}

.element-2 {
  @include
   absolute-center($position);
  color:blue;
}
```

sass

```
$position: 50%

@mixin absolute-center($position)
  position: absolute
  left: $position
  top: $position
  transform:translate(-50%,-50%)

.element-1
  @include absolute-center($position)

.element-2
  @include absolute-center($position)
  color:blue
```

Metalinguaggi: scss & sass

Functions: mixins with content box

css

```
.button {
  border: 1px solid black;
}
.button:not([disabled]):hover
{
  border-color: blue;
}
```

scss

```
@mixin hover-not-disabled {
  &:not([disabled]):hover {
    @content;
  }
}
.button {
  border: 1px solid black;
  @include hover-not-disabled {
    border-color: blue;
  }
}
```

sass

```
@mixin hover-not-disabled
  &:not([disabled]):hover
    @content

.button
  border: 1px solid black
  @include hover-not-disabled
    border-color: blue
```

Metalinguaggi: scss & sass

import

css

```
/* style.css */

body {
  color: #535353;
}
.dark-region {
  background-color:#000;
  color:#fff;
}
```

scss

```
/* src/_colors.scss */

$accent-color: #535353;
@mixin dark-background {
  background-color:#000;
  color:#fff;
}
```

```
/* style.scss */
@use ‘src/colors' as colors;
body {
  color: colors.$accent-color;
}
.dark-region {
  @include colors.dark-background;
}
```

sass

```
/* src/_colors.sass */

$accent-color: #535353
@mixin dark-background
  background-color:#000
  color:#fff
```

```
/* style.sass */
@use ‘src/colors' as colors
body
  color: colors.$accent-color

.dark-region
  @include colors.dark-background;
```

Metalinguaggi: scss & sass

Math: +,-,/,*,%

css

```
content {
  width: 100px;
}

.inner-content {
  width: 40px;
}

.outer-content {
  width: 160px;
}
```

scss

```
$content-width: 600px;

content {
  width:$content-width;
}

.inner-content {
  width: $content-width - 60px;
}

.outer-content {
  width: $content-width + 60px;
}
```

sass

```
$content-width: 600px

content
  width:$content-width

.inner-content
  width: $content-width - 60px

.outer-content
  width: $content-width + 60px
```

Ì

Metalinguaggi: scss & sass

Flow control: @if / @else, @each, @for

css

```
.element .bubble-1 {
  transition-delay: 0.3;
}
.element .bubble-2 {
  transition-delay: 0.6;
}
.element .bubble-3 {
  transition-delay: 0.9;
}
.element .bubble-4 {
  transition-delay: 1.2;
}
```

scss

```
@mixin theme($is-dark: false) {
  @if $is-dark {
   $sizes: 40px, 50px, 80px;
   @each $size in $sizes {
     .icon-#{$size} {
       font-size: $size;
       height: $size;
       width: $size;
     }
   }
  }
   @else {
     @for $i from 1 through 4 {
      .bubble-#{$i} {
        transition-delay: .3 * $i;
      }
   }
   }
}

.element {
 @include theme();
}
```

sass

```
@mixin theme($is-dark: false)
  @if $is-dark
   $sizes: 40px, 50px, 80px
   @each $size in $sizes
     .icon-#{$size}
       font-size: $size;
       height: $size;
       width: $size;

  @else
    @for $i from 1 through 4
     .bubble-#{$i}
       transition-delay: .3 * $i;

.element
  @include theme()
```

HTML & CSS

https://sass-lang.com/playground/#eJxVkMFqxDAMRO/5ioHuoVua0NAtlOTiX3ESeS3q2MV26LbL/nsVZwMtl

CORSO | Piano lezioni 2025... | contactcenter.indi... | Favicon Generator... | ICC board - Kanba... | Bando_CREATIVE-... | Tutti i preferiti

Playground

Copy URL

SCSS | Sass | v1.86.0

```scss
@mixin theme($is-dark: false) {
  @if $is-dark {
$sizes: 40px, 50px, 80px;
@each $size in $sizes {
  .icon-#{$size} {
    font-size: $size;
    height: $size;
    width: $size;
  }
}
}
  @else {
    @for $i from 1 through 4 {
  .bubble-#{$i} {
    transition-delay: .3 * $i;
  }
}
  }
```

Expanded | Compressed

CSS (Compiled)

```css
.element .bubble-1 {
  transition-delay: 0.3;
}
.element .bubble-2 {
  transition-delay: 0.6;
}
.element .bubble-3 {
  transition-delay: 0.9;
}
.element .bubble-4 {
  transition-delay: 1.2;
}
```

Console

Sass playground https://sass-lang.com/playground/

Progetto scss con note: apri il terminale e crea una nuova cartella per il progetto, ad esempio:

```
mkdir progetto-scss
cd progetto-scss
```

Inizializzazione del progetto con npm: all'interno della cartella del progetto, esegui:

```
npm init -y
```

Questo comando crea un file package.json con le impostazioni di default.

Installiamo sass: con il flag —save-dev

npm install sass —save-dev

Crea la struttura: organizzazione del progetto

```
tema-scss/
 ├── src/
 │    └── styles.scss
 ├── index.html
 └── package.json
```

Aggiungi scss: file styles.scss

```scss
// Temi
.light { @include theme(false); }
.dark  { @include theme(true); }

// Base
body {
  font-family: sans-serif;
  margin: 0;
  padding: 2rem;
}
button {
  border: none;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  cursor: pointer;
}
```

```scss
// Variabili
$light-bg: #fff;
$light-text: #000;
$dark-bg: #121212;
$dark-text: #fff;

// Mixin
@mixin theme($dark: false) {
  @if $dark {
    body {
      background: $dark-bg;
      color: $dark-text;
    }
    button {
      background: $dark-text;
      color: $dark-bg;
    }
  } @else {
    body {
      background: $light-bg;
      color: $light-text;
    }
    button {
      background: $light-text;
      color: $light-bg;
    }
  }
}
```

Aggiungi html: file index.html

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8" />
  <title>Tema SCSS Minimal</title>
  <link rel="stylesheet" href="dist/styles.css">
</head>
<body class="light">
  <button onclick="document.documentElement.classList.toggle('dark');
document.documentElement.classList.toggle('light')">
    Cambia Tema
  </button>
</body>
</html>
```

Aggiorniamo gli script: package.json

```
"scripts": {
  "build": "sass src/styles.scss dist/styles.css --no-source-map --style=compressed",
  "watch": "sass src/styles.scss dist/styles.css --watch"
}
```

Eseguiamo gli script: console

```
npm run build
```

LESS (Pre-processor)

- Usa le stesse idee di Sass (variabili, mixin, funzioni…).
- Meno usato oggi, ma ancora presente in progetti legacy (es: Bootstrap v3 usava LESS)
- Sintassi molto JS-like.

Esempio:

```
@primary: #007bff;

.button {
  background-color: @primary;
  &:hover {
    background-color: darken(@primary, 10%);
  }
}
```

PostCSS (Post-processor)

- È un tool/plugin engine più che un linguaggio
- Non ha una sua sintassi alternativa, lavora su CSS standard
- Usato per:
  - Autoprefixer (compatibilità browser)
  - PurgeCSS (rimozione classi inutilizzate)
  - PostCSS Nesting (simula nesting tipo SCSS)
  - TailwindCSS per aumentare capacità
- Altamente configurabile, basato su plugin

Riprendiamo il nostro progetto con SCSS: Installa i pacchetti necessari

```
npm install --save-dev sass postcss postcss-cli autoprefixer
```

Aggiungiamo il file di configurazione

postcss.config.js

```
module.exports = {
  plugins: [
    require('autoprefixer')
  ]
};
```

Aggiorniamo gli script in package.json

```
"scripts": {
  "build:scss": "sass src/styles.scss src/compiled.css",
  "build:postcss": "postcss src/compiled.css -o dist/styles.css",
  "build": "npm run build:scss && npm run build:postcss",
  "watch": "sass --watch src/styles.scss:src/compiled.css"
}
```

Aggiorniamo la lista dei browser target in package.json

```
"browserslist": [
  "last 2 versions",
  "IE 11"
]
```

Usiamo una proprietà scss che abbia bisogno di prefissi

```
button {
  border: none;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  cursor: pointer;
  display: flex;
}
```

Eseguiamo il comando

```
npm run build
```

Vuoi usare sia Tailwind che Nesting?

npm install -D tailwindcss postcss-cli autoprefixer postcss-nested (versione 3.x.x tailwind)
npx tailwindcss init

```
module.exports = {
  plugins: [
    require(‘tailwindcss'),
    require('postcss-nested'),
    require('autoprefixer')
  ]
};
```

```
"scripts": {
  "build": "postcss src/styles.css -o dist/styles.css",
  "watch": "postcss src/styles.css -o dist/styles.css --watch"
}
```

Esempio

```
@tailwind base;
@tailwind components;
@tailwind utilities;

.btn {
  @apply px-4 py-2 rounded bg-blue-500 text-white;

  &:hover {
    @apply bg-blue-600;
  }
}
```